

Pipeline Hệ Thống Giám Sát An Toàn Nhà Máy

Kiến trúc chi tiết end-to-end cho hệ thống 2 camera: Re-ID · PPE · Vùng cấm
· Phát hiện ngã — từ tầng thu hình đến dashboard React, kèm phân bổ VRAM, ngân sách độ trễ real-time và topology hạ tầng triển khai.

Thiết bị: Laptop · GPU 4GB VRAM

Nguồn: 2 camera USB

Inference: Triton 24.01 · gRPC · 100% model

Backend: FastAPI

Frontend: React 19 + Vite

Storage: R2 (ảnh) + PostgreSQL

PHẠM VI HỆ THỐNG

4 tính năng nghiệp vụ

Person Re-ID

Định danh nhân viên bằng vector đặc trưng OSNet 512-D, gallery **dùng chung giữa 2 camera** — cùng một người đi qua cam nào cũng ra cùng ID.

✓ CHẠY TRÊN TRITON

PPE Detection

4 classifier kiểm tra mũ, kính, găng tay, áo phản quang trên các crop bộ phận cơ thể, chu kỳ 2s/người.

✓ CHẠY TRÊN TRITON

Restricted Zone

Phát hiện người đi vào vùng cấm: kiểm tra **điểm chân** (đáy bbox) so với polygon vẽ trên frontend. Thuần CPU, chạy mỗi frame.

+ XÂY MỚI

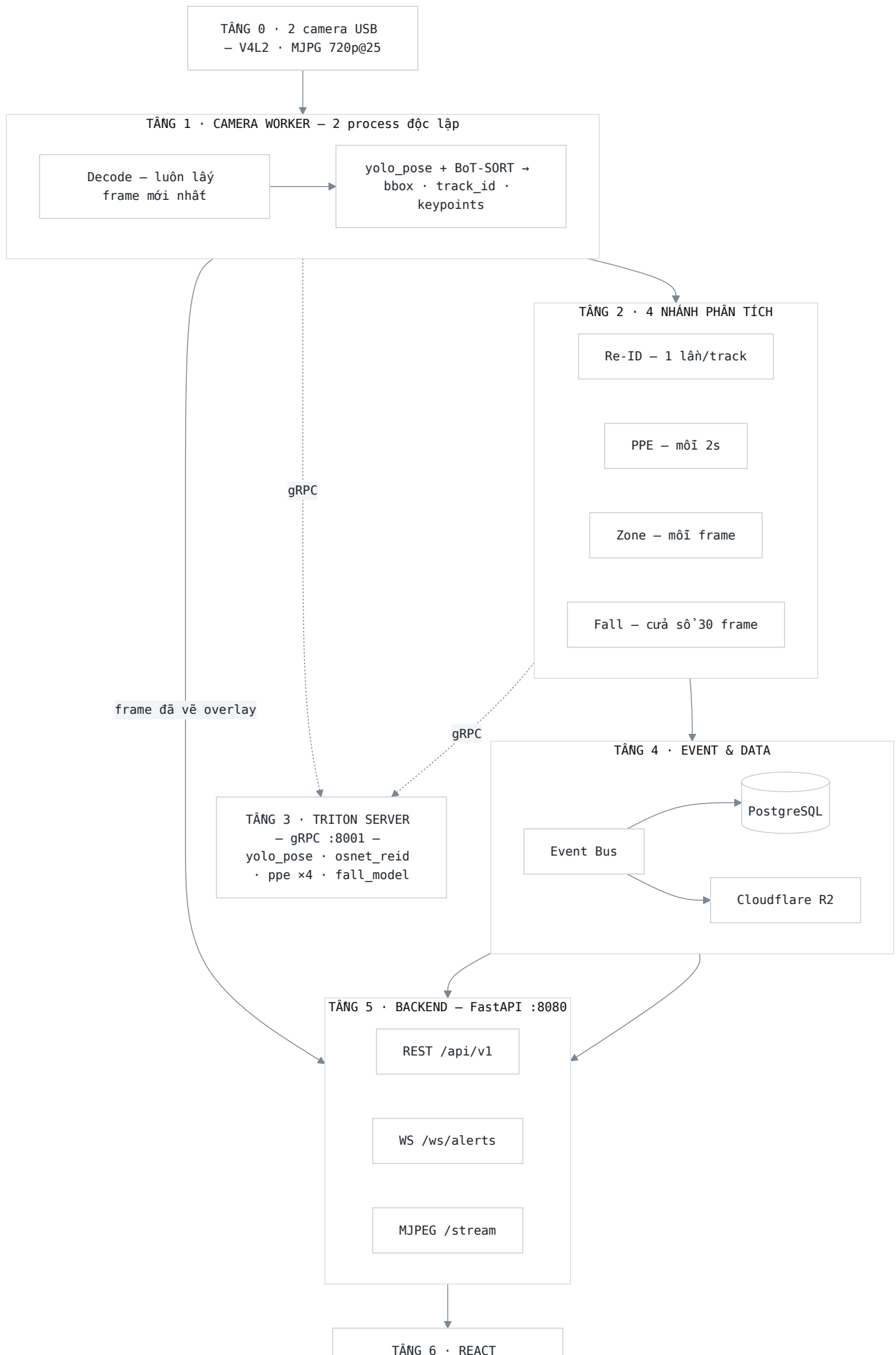
Fall Detection

Nhận diện người ngã từ chuỗi keypoints (đã có sẵn từ YOLOv8-Pose). Kiến trúc chừa sẵn slot Triton cho model.

+ CHỜ MODEL

Pipeline end-to-end

Mỗi camera chạy một **process riêng** (thoát GIL hoàn toàn), mọi model nặng dồn về một Triton Server duy nhất. Sự kiện vi phạm chảy một chiều về Event Bus → Database → WebSocket → Dashboard.



Nét liền = luồng dữ liệu chính (trên xuống) · Nét đứt = gọi inference sang Triton. Mọi model nằm gọn trong một khối Triton duy nhất thay vì nối riêng từng model.

TẦNG 0 – INGEST

Thu hình từ 2 camera USB

OpenCV VideoCapture
backend V4L2 (Linux)

USB camera ×2
/dev/video0 · /dev/video2

FOURCC MJPG · 720p@25
tránh nghẽn băng thông USB

- Mỗi camera một device index riêng, cấu hình trong DB (bảng `cameras`: device, tên, vị trí, zones). Kiểm tra thiết bị bằng `v4l2-ctl --list-devices`.
- **Bắt buộc ép FOURCC MJPG + 1280×720**: 2 luồng YUYV thô sẽ vượt băng thông USB — nguyên nhân số 1 khiến camera thứ hai không mở được. Nếu vẫn nghẽn, cắm 2 cổng thuộc 2 USB controller khác nhau.
- **Luôn đọc frame mới nhất** — nếu xử lý chậm hơn 25fps thì bỏ frame cũ, không để dồn buffer gây trễ dây chuyền.
- Tự động mở lại khi camera bị rút/cắm lại, báo trạng thái `online/offline` lên dashboard (UI đã có nút power + đèn trạng thái).

TẦNG 1 – DETECTION & TRACKING

Camera Worker: process riêng cho mỗi camera

multiprocessing
1 process/camera – thoát GIL

yolo_pose trên Triton
ONNX FP16 · dynamic batch 2 cam

BoT-SORT (BoxMOT)
tracker thuần Python, độc lập model

Mỗi camera một process độc lập — thoát hoàn toàn GIL của Python. YOLOv8n-Pose chạy trên Triton (đã chốt) nên worker **không giữ model nào**: chỉ decode, tiền/hậu xử lý và gọi gRPC. Worker là thành phần quyết định độ trễ end-to-end — mọi bước

trong vòng lặp frame phải có thời gian chặn xác định, **tuyệt đối không chờ I/O mạng** (DB, R2, email) bên trong vòng lặp.

- Chuỗi xử lý mỗi frame: preprocess (letterbox 640) → gRPC `yolo_pose` → decode output + NMS → **BoT-SORT của thư viện BoxMOT** nhận detections và gán `track_id`.
- BoxMOT hoạt động trên detections thuần (xyxy, conf) nên không phụ thuộc framework model; **tắt Re-ID nội bộ của tracker** (appearance-free) — định danh đã có nhánh Re-ID riêng lo.
- Đầu ra mỗi frame: `bbox · track_id · conf · 17 keypoints`; track ID tiền tố theo camera (`cam1-17`) — **danh tính toàn cục do Re-ID quyết định**, không phải tracker.
- Worker vẽ overlay (bbox màu, tên, cảnh báo) và đẩy frame ra shared memory cho MJPEG streamer; UI có toggle bật/tắt overlay.

Vì sao dồn pose lên Triton: 2 camera dùng chung 1 instance `yolo_pose` — dynamic batching gộp frame 2 cam thành một batch GPU, VRAM không tăng khi thêm camera; đồng thời toàn bộ chi phí GPU quy về một điểm quan sát duy nhất (Triton metrics :8002).

TẦNG 2 – ANALYTICS

4 nhánh phân tích song song

Re-ID — định danh

TẦN SUẤT: 1 lần khi track mới xuất hiện · GPU qua Triton

- Crop người → chuẩn hóa 128×256 → `osnet_reid` (gRPC) → vector 512-D, L2-norm
- So cosine với **gallery toàn cục** (nạp từ bảng `persons`) — threshold 0.3
- Gallery chạy trong **Identity Service dùng chung**: cả 2 camera cùng hỏi một chỗ → cross-camera re-identification

PPE — kiểm tra bảo hộ

TẦN SUẤT: mỗi 2s / người · GPU qua Triton

- Từ keypoints cắt 4 vùng: đầu, mặt, thân, tay
- 4 crop → 4 model classify 128×128 trên Triton (dynamic batching gộp request từ cả 2 cam)
- Vi phạm → sự kiện `ppe_violation` + cooldown 30s chống ghi trùng và spam cảnh báo

- Trùng → gán `person_id` ; không trùng
→ tạo person mới + lưu vector

Zone — vùng cấm

TẦN SUẤT: mỗi frame · thuần CPU, ~0ms

- Điểm chân = trung điểm cạnh đáy bbox (hoặc keypoints cổ chân nếu tin cậy)
- `cv2.pointPolygonTest` với các polygon của camera đó (bảng `zones`, vẽ từ frontend)
- Debounce N frame liên tiếp trong vùng mới bắn cảnh báo — tránh nhiễu tracker
- Sự kiện `zone_intrusion` kèm `person_id` (nếu đã định danh) + ảnh bằng chứng

Fall — phát hiện ngã

TẦN SUẤT: cửa sổ trượt keypoints · slot Triton chờ sẵn

- **Đã chốt: model nhận chuỗi keypoints**
— buffer trượt theo **timestamp** từng track, resample về đúng FPS model được train (không dùng số frame cứng vì FPS inference dao động); dữ liệu có sẵn miễn phí từ Tầng 1
- Interface
`FallDetector.predict(keypoints_window)`
→ `score` — model cắm vào là chạy, không đụng phần còn lại
- Model chuỗi keypoints (LSTM/GCN) rất nhẹ — dự kiến chỉ ~100–200MB VRAM trên Triton
- Dự phòng chạy được ngay: heuristic tỷ lệ bbox W/H > 1.4 + góc thân so phương đứng
- Sự kiện `fall_detected` — mức CRITICAL, bỏ qua cooldown

TẦNG 3 — INFERENCE

Triton Inference Server & ngân sách 4GB VRAM

Triton 24.01

Docker · ONNX Runtime backend

gRPC :8001

overhead 2–10ms đã đo

Dynamic Batching

gộp request 2 camera

FP16

giảm ~50% VRAM model

Toàn bộ model — kể cả YOLO-Pose — nằm trên một Triton Server phục vụ cả 2 camera: **Python không giữ model nào, thêm camera không tốn thêm VRAM cho model**, chỉ tăng lượng request được dynamic batching gộp lại.

Ngân sách VRAM (ước tính, cần đo thực tế bằng nvidia-smi)

THÀNH PHẦN	NƠI CHẠY	PRECISION	VRAM ƯỚC TÍNH	GH
CUDA context + Triton runtime	Triton	—	~500 MB	Ch đị
yolo_pose (1 instance chung 2 cam)	Triton	FP16	~400–600 MB	D b đ c T
osnet_reid	Triton	FP32→FP16	~250 MB	m 16 g
ppe_head/face/hand/torso (×4)	Triton	FP32→FP16	~400 MB	M c n
fall_model (chuỗi keypoints)	Triton	FP16	~100–200 MB	M c k lý
Tổng dự kiến			~1.7–2.0 GB / 4 GB	H ~2 r m

Quy tắc an toàn VRAM: mọi model trên Triton để instance_count: 1, xuất ONNX FP16. Nhờ dồn toàn bộ model về Triton, ngân sách dư ~2GB — đủ chỗ cho model fall và cả phương án nâng pose lên YOLOv8s sau này nếu cần độ chính xác cao hơn.

Event Bus

multiprocessing.Queue → backend

PostgreSQL 16

metadata – container trong compose

Cloudflare R2

ảnh/video bằng chứng – S3 API (boto3)

SQLAlchemy 2.0 + Alembic

ORM + migration

Đã chốt tách đôi lưu trữ: media (ảnh/video bằng chứng) đẩy lên **Cloudflare R2**, dữ liệu có cấu trúc nằm trong database. DB chỉ lưu `evidence_key` (object key trên R2), tuyệt đối không lưu ảnh trong DB.

- Mọi nhánh phân tích chỉ **phát sự kiện**, không tự ghi DB — một consumer duy nhất ghi DB và đẩy ảnh vào hàng đợi upload.
- **Uploader R2 bất đồng bộ:** pipeline không bao giờ chờ mạng — ảnh spool tạm ra đĩa (`evidence_spool/`), worker riêng upload với retry; mất mạng không mất bằng chứng, upload xong xóa file tạm.
- **PostgreSQL** được chọn vì hệ có nhiều process cùng ghi (event consumer + backend API); chạy container ngay trong compose có sẵn.
- Schema: `persons` (thêm tên/ảnh đại diện), `violations` (hợp nhất PPE + zone + fall: `camera_id`, `type`, `severity`, `evidence_key`), `cameras`, `zones` (polygon JSON), `users`.

TẦNG 5 – SERVING

Backend API — cầu nối AI ↔ Dashboard

FastAPI + Uvicorn

:8080

WebSocket

đẩy cảnh báo <100ms

MJPEG

multipart/x-mixed-replace

JWT Auth

login/register đã có UI

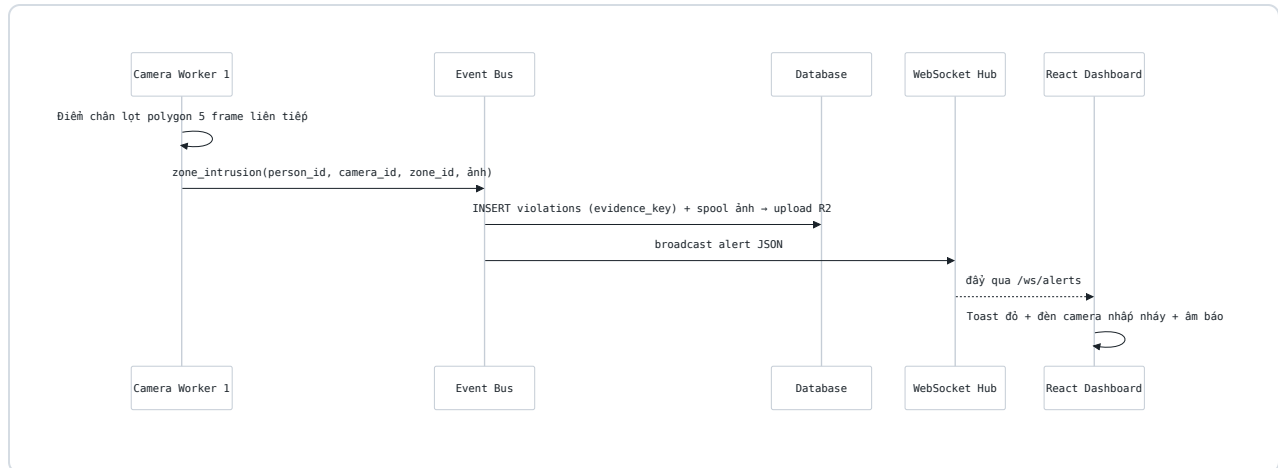
Pydantic v2

schema chung types với FE

- **REST** `/api/v1/`: cameras (CRUD), zones (CRUD polygon), violations (filter + phân trang, ảnh bằng chứng trả về dạng **presigned URL R2** hết hạn 1h), persons, reports (thống kê theo ngày/loại), auth (JWT), settings (ngưỡng runtime, bật/tắt model per-camera), alerts/emergency.

- **WebSocket** `/ws/alerts` : fan-out sự kiện real-time cho mọi client đang mở dashboard — pattern ConnectionManager chuẩn của FastAPI.
- **MJPEG** `/stream/{camera_id}` : stream frame đã vẽ overlay từ shared memory của worker. Đủ tốt cho 2 cam trên LAN; nâng cấp WebRTC (MediaMTX) là bước sau nếu cần độ trễ <200ms.

Chuỗi sự kiện một cảnh báo vùng cấm



TẦNG 6 – FRONTEND

React Dashboard — VisionGuard AI

React 19 + Vite 8

SPA · CSS Modules · theme sáng/tối

React Router 7

8 trang

TanStack Query

sẽ thêm – fetch REST

WebSocket client

sẽ thêm – alerts real-time

UI đã dựng hoàn chỉnh và chạy được. Mỗi khối dữ liệu trên giao diện ánh xạ thẳng vào một nguồn ở Tầng 5:

- **Dashboard** — 4 thẻ metrics (camera hoạt động · model · vi phạm · system health), khung live feed, cột "Recent Violations" → nguồn: REST tổng hợp + MJPEG + WebSocket.
- **Cameras** — grid camera (thêm/xóa/bật tắt nguồn), toggle AI overlay, modal zoom có **telemetry FPS/độ trễ/CPU** + nhật ký phân tích + checkbox bật tắt model theo từng cam → cần Runtime Settings API + kênh telemetry.

- **Violations** — bảng log, tìm kiếm, lọc severity, modal snapshot → ảnh lấy qua presigned URL R2.
- **Reports** — 4 KPI (tổng vi phạm · tỷ lệ tuân thủ PPE · báo động sai · thời gian phản hồi) + biểu đồ tuần + xuất CSV/PDF.
- **Settings** — theme, email báo cáo hằng ngày 18:00, SMS khẩn cấp, auto-record 10s, số ngày lưu snapshot → mỗi toggle tương ứng một dịch vụ phụ trợ (bảng bên dưới).
- **Emergency Alert** (nút sidebar) — phát cảnh báo thủ công toàn hệ thống.
- **Login/Register** — nối JWT auth.

FE còn thiếu so với kiến trúc: zone editor vẽ polygon trên khung hình (tính năng vùng cấm), loại vi phạm Fall, trang quản lý nhân viên (gallery Re-ID), route /models chưa đăng ký (bấm thẻ "Connected Model" ra trang trắng), và biểu đồ Reports chưa render cột (bug CSS nhỏ).

REAL - TIME

Ngân sách độ trễ & nguyên tắc thời gian thực

Hệ có hai đường dữ liệu với yêu cầu khác nhau: **đường video** (người vận hành nhìn) chấp nhận trễ vài trăm ms; **đường cảnh báo** phải tới dashboard gần như tức thời. Tách riêng hai đường này là lý do dùng WebSocket cho alert thay vì nhét vào luồng video.

CHẶNG	ĐỘ TRỄ ƯỚC TÍNH	GHI CHÚ
Capture USB 720p@25	40 ms/frame	Chu kỳ frame — luôn lấy frame mới nhất
Glass-to-glass (đường video)	~200–400 ms	Đạt yêu cầu giám sát công nghiệp
Phát hiện → toast (đường cảnh báo)	< 150 ms	Sau khi debounce/cooldown đạt điều kiện

CHẶNG	ĐỘ TRỄ ƯỚC TÍNH	GHI CHÚ
Preprocess (letterbox, normalize)	~2–4 ms	NumPy trong process worker
yolo_pose trên Triton (gRPC)	~10–25 ms	Gồm max_queue_delay 5ms của dynamic batching
BoT-SORT tracking	~3–8 ms	CPU, tăng theo số người trong khung
Zone check + overlay + JPEG encode	~5–10 ms	CPU
MJPEG tới browser (LAN)	~100–300 ms	Buffer multipart của browser
Glass-to-glass (đường video)	~200–400 ms	Đạt yêu cầu giám sát công nghiệp
Phát hiện → toast (đường cảnh báo)	< 150 ms	Sau khi debounce/cooldown đạt điều kiện

4 nguyên tắc giữ hệ chạy real-time

- **Latest-frame-only:** hàng đợi giữa capture và xử lý tối đa 1 phần tử — xử lý chậm thì bỏ frame cũ, không bao giờ xếp hàng. Kèm ràng buộc **tối đa 1 request pose in-flight/worker** → Triton không bao giờ backlog quá 2 request pose. FPS có thể giảm nhưng độ trễ không bao giờ phình.
- **Không I/O chặn trong vòng lặp frame:** ghi DB, upload R2, gửi email/SMS đều đi qua Event Bus sang process khác xử lý.
- **Nhánh chậm chạy bất đồng bộ:** Re-ID và PPE trả kết quả trễ vài trăm ms cũng không ảnh hưởng FPS — kết quả gắn vào track khi xong, hiển thị ở frame kế tiếp.
- **Đo liên tục:** mỗi chặng phát telemetry (FPS, ms từng bước) → modal camera trên dashboard hiển thị trực tiếp; phía GPU theo dõi qua Triton metrics :8002.

Điểm nghẽn của gRPC nội bộ không nằm ở mạng (loopback chịu hàng GB/s) mà ở **serialize + copy bộ nhớ**: tensor pose FP32 640×640 nặng 4.9MB/frame — 2 cam @25fps là ~**245MB/s bị copy 3-4 lần**, trả giá bằng CPU. Re-ID và PPE payload nhỏ, gọi thưa — gRPC thuần là đủ. Các quyết định dưới đây triệt tiêu dần chi phí trên đường pose.

#	QUYẾT ĐỊNH	CHI TIẾT
1	Latest-frame + 1 in-flight	Buffer camera 1 phần tử (ghi đè); mỗi worker tối đa 1 request pose đang chờ → không bao giờ hình thành backlog
2	Dynamic batching batch 2	<code>max_batch_size 2 · preferred [2] · queue_delay 2000µs · instance 1</code> — queue delay là điểm khởi đầu, tune bằng <code>perf_analyzer (0/1/2/5ms)</code>
3	Không sync tick 2 worker	Chỉ đặt cùng FPS mục tiêu, để dynamic batching tự ghép — tránh coupling giữa 2 process. Chỉ xem lại khi tỷ lệ batch-2 thấp (metrics :8002) VÀ GPU là bottleneck
4	Fall theo timestamp	Buffer keypoints đánh timestamp, resample về đúng FPS model fall được train — không dùng số frame cứng
5	Ưu tiên bằng tần suất tự nhiên	Re-ID 1 lần/track · PPE mỗi 2s · fall theo window → không tranh GPU với pose. Chưa dùng <code>priority_levels</code> (chỉ có tác dụng trong queue của 1 model, không cross-model)
6	Ladder tối ưu	uint8 + normalize trong ONNX graph (làm ngay từ export script, payload giảm 4×) → ONNX FP16 + batch 2 → TensorRT FP16 (build .plan trên đúng máy đích, đúng version container, verify accuracy) → giảm size/FPS inference

Config yolo_pose đã chốt

```
max_batch_size: 2

dynamic_batching {
  preferred_batch_size: [ 2 ]
  max_queue_delay_microseconds: 2000
}
```

```
instance_group [ { count: 1, kind: KIND_GPU } ]
```

Giao thức đo — không tối ưu chạy

- **perf_analyzer** (công cụ chính thức của Triton): quét queue delay 0/1/2/5ms, đọc p95/p99 latency và throughput từng cấu hình.
- **Triton metrics :8002**: tỷ lệ batch thực tế, queue time, GPU utilization theo từng model.
- **Telemetry per-stage trong worker**: decode / preprocess / gRPC / tracking / encode — hiển thị trực tiếp trên modal camera của dashboard.
- **Test sustained 1-2 giờ** trước khi kết luận — laptop throttle nhiệt khiến benchmark gần đẹp một cách giả tạo.

Vì sao chưa bật shared memory ngay: uint8-in-graph giảm payload 4× (4.9MB → 1.2MB/frame) với chi phí gần bằng 0. **Triton System Shared Memory** (đăng ký vùng POSIX shm, gRPC chỉ gửi tên vùng nhớ — còn đúng 1 lần copy host → GPU) là bước kế tiếp đã nằm sẵn trong thiết kế, chỉ bật khi telemetry chỉ ra chi phí copy vẫn đáng kể. Với container: mount chung /dev/shm trong compose.

HẠ TẦNG

Topology triển khai trên laptop

Docker Compose

triton + postgres (+ backend, frontend sau)

NVIDIA Container Toolkit

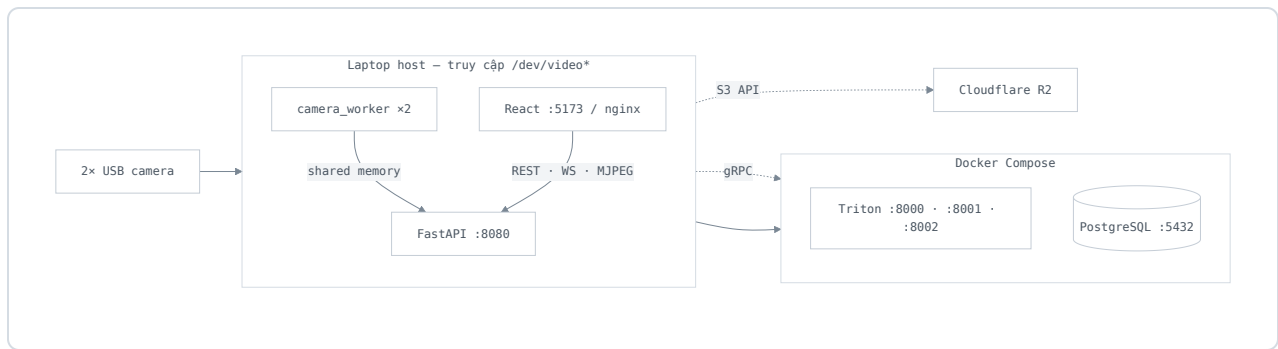
GPU passthrough cho Triton

Triton metrics :8002

Prometheus format

.env

secrets R2/DB/SMTP — không commit



- **Worker + backend chạy trên host** (chưa container hóa giai đoạn này): cần truy cập trực tiếp `/dev/video*` và chia sẻ frame qua shared memory với backend — nhanh và đơn giản hơn bridge qua network.
- **Triton trong container** với NVIDIA runtime, `shm_size 2GB`, mount `triton_model_repo/`; healthcheck `/v2/health/ready`; `restart: unless-stopped`.
- **PostgreSQL trong container**, volume riêng cho data; backup bằng `pg_dump` định kỳ.
- **Quan sát hệ thống**: Triton xuất Prometheus metrics ở `:8002` (GPU utilization, queue time, batch size từng model) — khi cần soi hiệu năng chỉ việc gắn Prometheus + Grafana, không sửa code.
- **Khởi động đúng thứ tự**: compose up (Triton + Postgres) → đợi healthcheck → backend → camera workers → frontend. Worker tự retry gRPC khi Triton chưa sẵn sàng.

DỊCH VỤ PHỤ TRỢ

Yêu cầu rút ra từ UI thực tế

Các toggle và nút đã tồn tại trên giao diện đòi hỏi những dịch vụ nền sau — bổ sung vào backend, không ảnh hưởng pipeline AI:

TÍNH NĂNG TRÊN UI	DỊCH VỤ NỀN CẦN XÂY	CÔNG NGHỆ
Email báo cáo vi phạm hằng ngày (18:00)	Job tổng hợp vi phạm trong ngày + gửi mail	APScheduler + SMTP
SMS cảnh báo khẩn cấp khi Danger	Notification hook nghe Event Bus	Twilio / eSMS API


```
|— docker-compose.yml      # triton + postgres + backend + frontend(nginx)
|— config.yaml             # Cấu hình tập trung (camera, ngưỡng, model)
```

ĐÃ CHỜT · 17/07/2026

4 quyết định kiến trúc

Q1 Camera: USB. V4L2 + ép FOURCC MJPG 720p@25 để 2 cam không nghẽn băng thông USB — chi tiết Tầng 0.

Q2 YOLOv8-Pose: chạy trên Triton. Worker Python không giữ model nào; tracking dùng BoT-SORT của thư viện BoxMOT trên detections — chi tiết Tầng 1. VRAM tổng ~1.7–2.0GB.

Q3 Fall model: nhận chuỗi keypoints. Buffer trượt ~30 frame/track, interface `FallDetector.predict(keypoints_window)` chờ sẵn — chi tiết Tầng 2.

Q4 Lưu trữ tách đôi: Cloudflare R2 cho ảnh/video, PostgreSQL cho dữ liệu có cấu trúc. DB chỉ giữ object key; uploader bất đồng bộ có pool chống mất mạng — chi tiết Tầng 4.

THAM KHẢO

Repo đã khảo sát

[PPE-Safety-Compliance-Detection-System](#) — YOLOv8 + Re-ID + DeepSORT, stack FastAPI + React + PostgreSQL giống mục tiêu của mình nhất

[PPE-Detection-and-Danger-Zone-Monitoring-System](#) — mẫu tính năng danger zone vẽ polygon

[triton-server-yolo](#) · [triton-client-yolo](#) — chuẩn config.pbtxt, ONNX dynamic batch cho YOLO trên Triton

[BoxMOT](#) — BoT-SORT/ByteTrack thuần Python, độc lập model

[yolov8-pose-fall-detection](#) · [Human-Fall-Detection](#) — fall detection từ keypoints: tỷ lệ bbox, góc thân

Savant (insight-platform) — framework công nghiệp trên DeepStream; tham khảo kiến trúc, không dùng vì overkill cho 2 camera/4GB

KIẾN TRÚC v1.3 · 2 CAMERA USB · 4GB VRAM · 100% MODEL TRÊN TRITON · R2 + POSTGRESQL · GLASS-TO-GLASS ~200–400MS · SERVING ĐÃ CHỐT — docs/ARCHITECTURE.md trong repo là bản Mermaid tương ứng